

Deep Generative Models

Lecture 1



Jan Čech

Visual Recognition Group

Department of Cybernetics

Faculty of Electrical Engineering, Czech Technical University in Prague

Deep Generative Models



- Models generating samples from a given distribution
 - The target distribution can be very complex (high-dimensionality)
 - Synthesis of **realistic** samples of, e.g. Text, Code, Audio, Music, **Images**, Video, 3D models
- Unconditioned / Conditioned models
 - Conditioning by a “prompt” (text, image, semantic maps, ...)
- **Three lectures:**
 1. Intro, Brief taxonomy and Evaluation, VAEs, GANs
 2. Normalizing flows, Diffusion Models, Comparisons
 3. Deepfake detection



Examples

Examples of Deep Generative Models



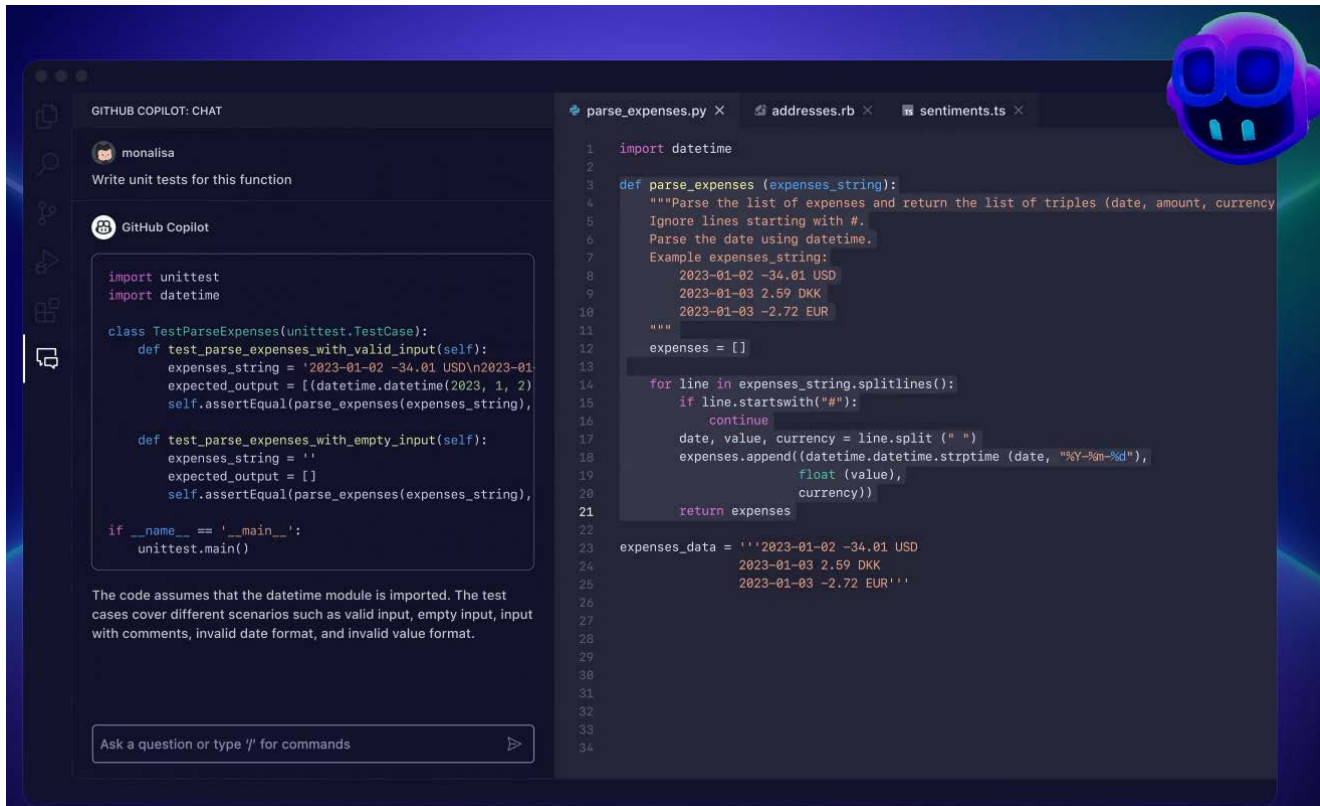
■ Text



- Large Language models
- GPT (OpenAI), Gemini (Google), Llama (Meta), Deepseek – open source, ...

■ Code

- GitHub Copilot – free for students, ...





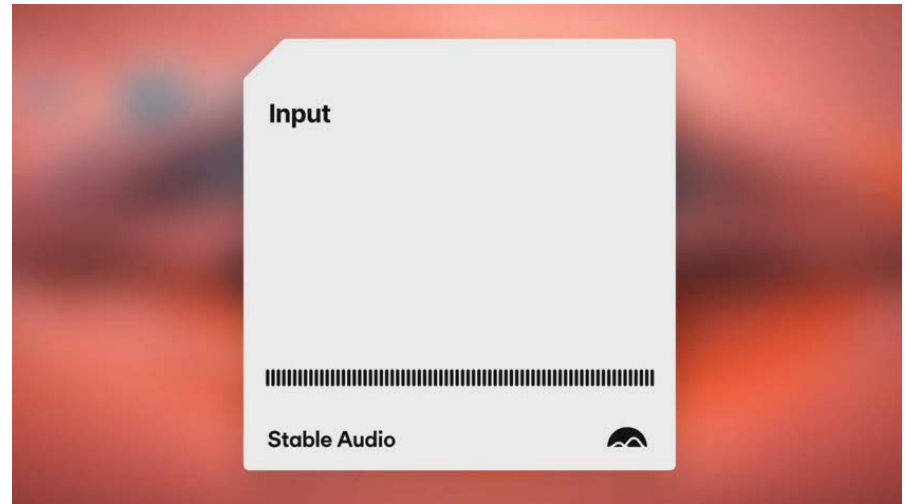
Examples of Deep Generative Models

■ Audio

– text2audio, audio2audio

- [Stable Audio](#) (Stability.AI)

Prompt: Epic music, big drums, strong melody



- [Suno AI](#) (Cambridge, MA, USA)

Lyrics: Computer Vision Method course focuses on the following computer vision problems: finding correspondences between images using image features and their robust invariant descriptors, image retrieval, object detection and recognition, and visual tracking.

Style: Large symphonic orchestra and children chorus, epic melody.



– text2speech

- Many models, emotional speech, voice cloning, very realistic...

Examples of Deep Generative Models

- Unconditioned Image generators
 - Generating photo-realistic samples from image distributions



(Images synthesized by a random sampling)

Examples of Deep Generative Models

- Text2image
 - DALL-E (OpenAI), Imagen (Google), Midjourney, ...
 - **StableDiffusion** (Stability.AI, free open source)



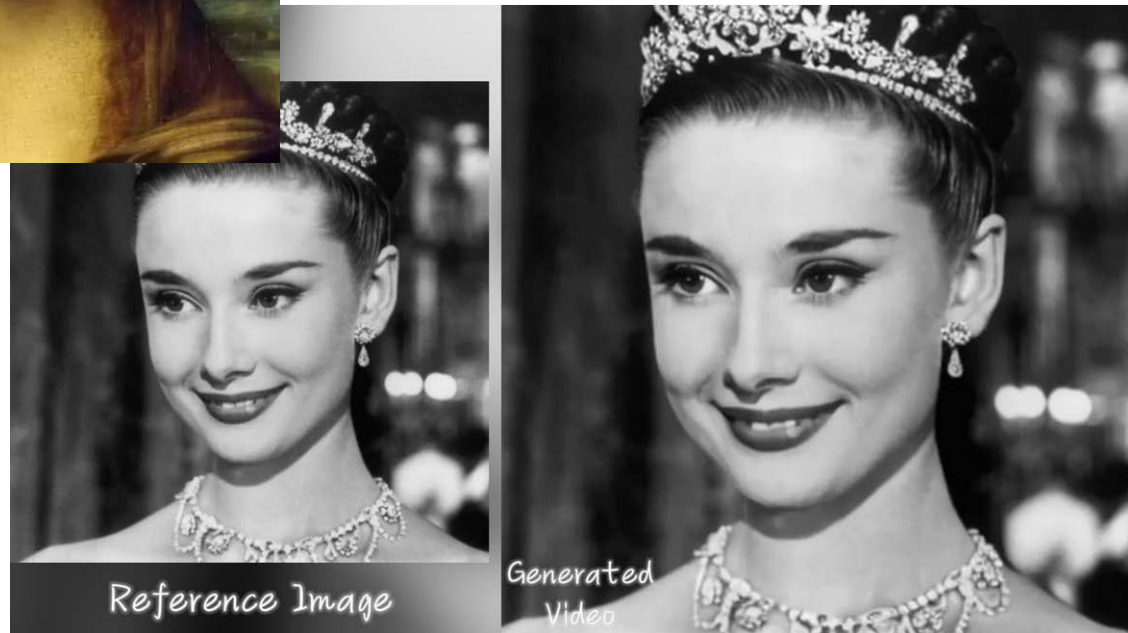
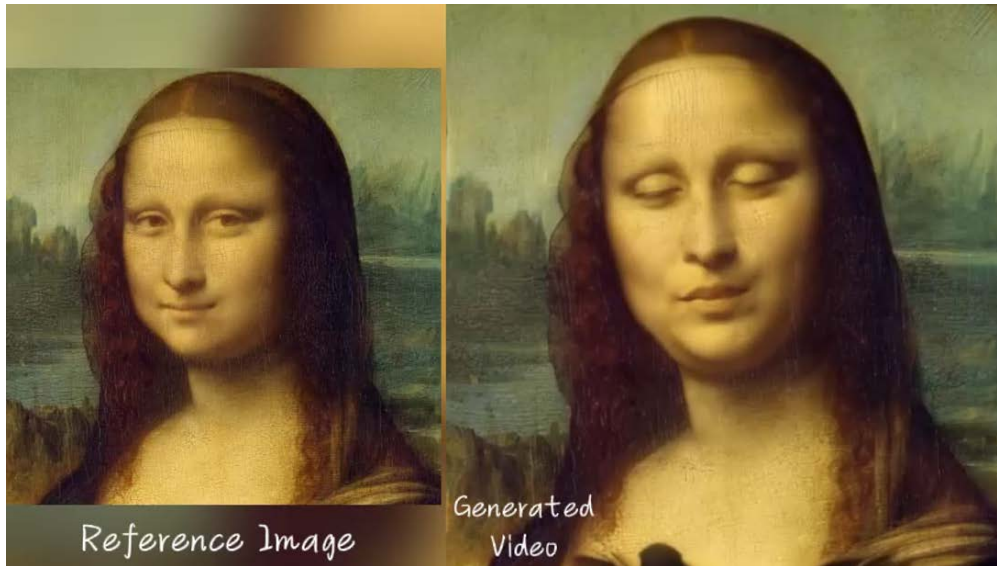
panda mad scientist mixing sparkling chemicals, artstation



a propaganda poster depicting a cat dressed as french emperor
napoleon holding a piece of cheese

Examples of Deep Generative Models

- “Talking/Singing head” (audio2video)
 - EMO: Emote Portrait Alive [[Tian-2024](#)] (Alibaba group)



Examples of Deep Generative Models

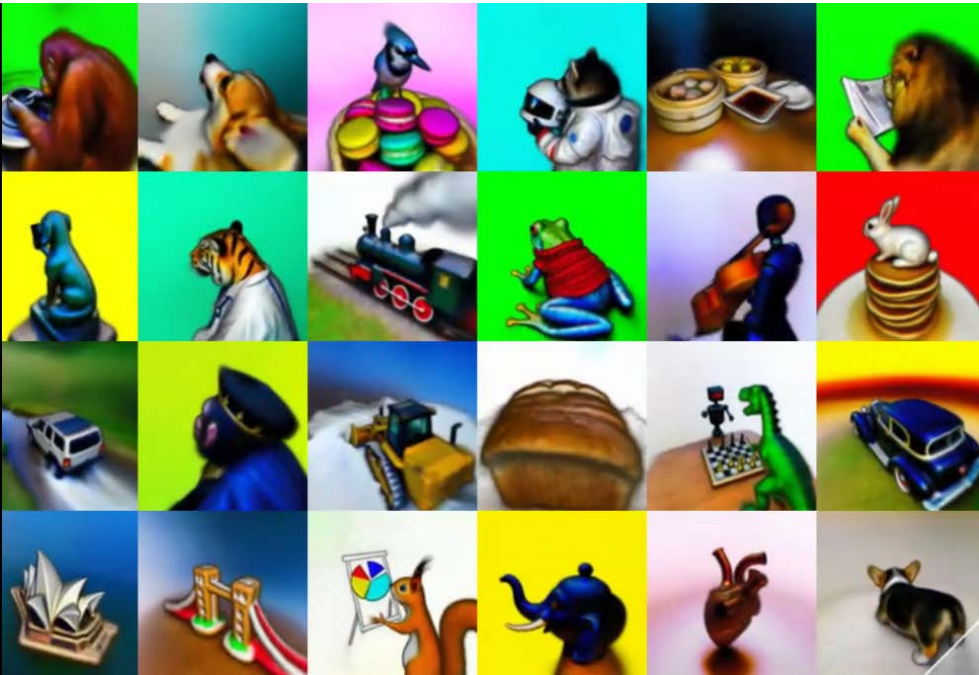
- Fully synthetic AI avatars
 - E.g., Synthesia ~2024



Examples of Deep Generative Models

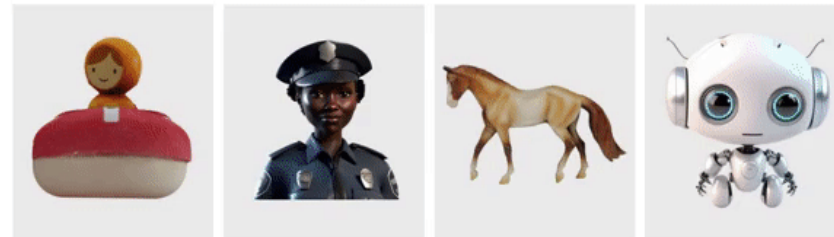
- 3D models (audio2video)
 - Text to 3D [[DreamFusion-2022](#)] (Google), [[Magic3D-2023](#)] (NVIDIA)
 - Image to 3D (TripoSR, Stability.AI)

DreamFusion

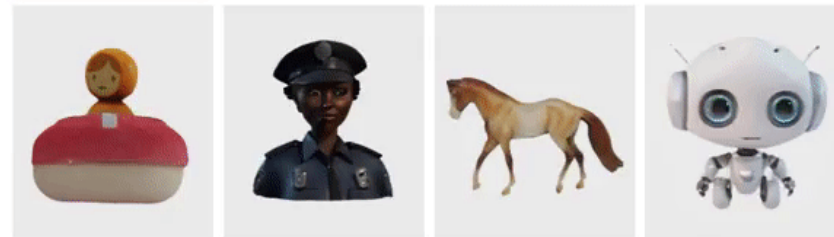


TripoSR

Input images



TripoSR outputs



Examples of Deep Generative Models

- Video
 - Text2video, image2video, video2video
 - [SORA](#) (OpenAI)

A litter of golden retriever puppies playing in the snow. Their heads pop out of snow



Beautiful, snowy Tokyo city is bustling. The camera moves through the bustling city street, following several people enjoying the beautiful snowy weather and shopping at nearby stalls. Gorgeous sakura petals are flying through the wind along with snowflakes



Examples of Deep Generative Models

- Video – long shots with audio
 - E.g., VEO3 (Google) May, 2025,
 - Text2video



- Open-weights models appear
 - Not as good, but are getting better



Warm up: A shallow model

A shallow generative model

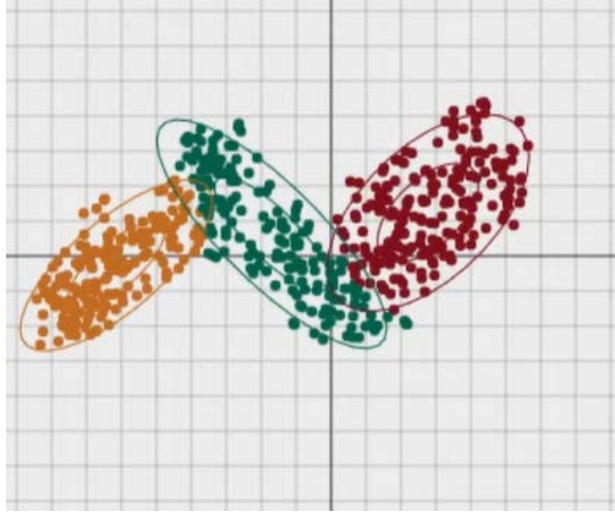


Figure credit: Stephano Ermon

Training set:

$$\text{TS} = \{x_1, \dots, x_n\}$$

$$x_i \sim p(x)$$

■ Gaussian mixture

$$p(x) = \sum_z p(x, z) = \sum_z p(z) p(x | z)$$

$$p(x) = \sum_k p_k \mathcal{N}(\mu_k, \Sigma_k)$$

- Fit the model ~ estimate parameters by maximum likelihood
- Generate new data from the model



Brief Taxonomy and Evaluation



Taxonomy of Deep Generative Models

- Several approaches:

1. **Variational Autoencoders** [[Kingma-2014](#)]
2. **Generative Adversarial Networks (GANs)** [[Goodfellow-2014](#)]
3. Autoregressive models [[Oord-2016](#)]
4. Normalizing flow models [[Dinh-2017](#)]
5. Diffusion models [[Sohl-Dickstein-2015](#), [Rombach-2022](#)]

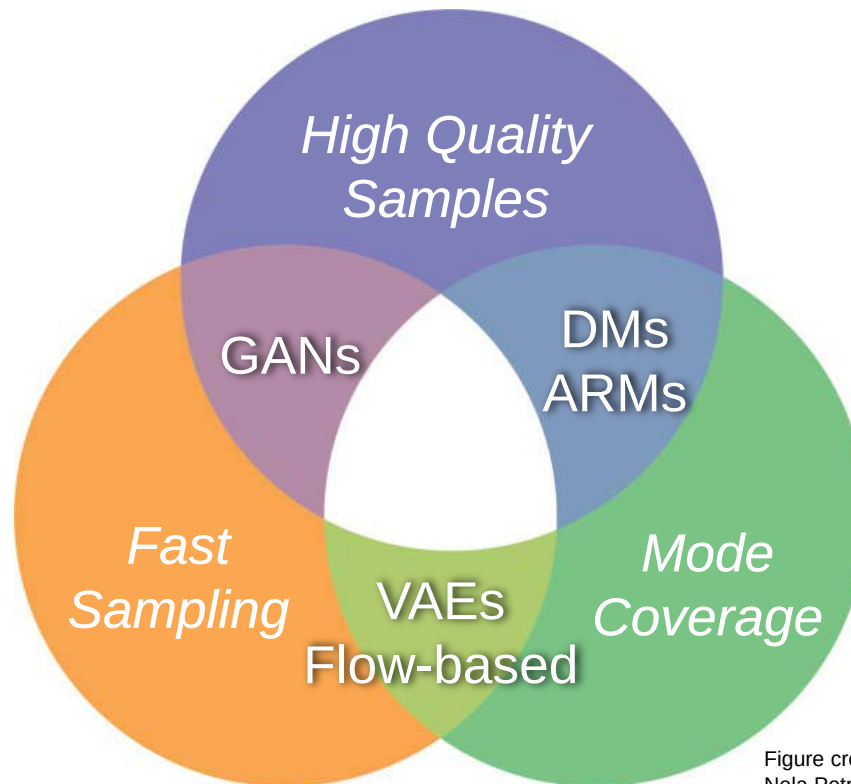


Figure credit
Nela Petzelkova



How to Measure Quality of Generative Models?

- Synthetic samples should resemble samples of real distribution in the sense of:
 - Fidelity (no obvious artifacts visible)
 - Diversity (enough variability respecting the original distribution)
 - Mode collapse – model always generate the same sample
- All methods use classifier trained on ImageNet (usually Inception v3)

1. Inception score (IS) [[Salimans-2016](#)]

- Only synthetic dataset
- Output softmax score $p(y|x)$

$$\text{IS} = \exp\left(\mathbb{E}_x \text{KL}(p(y|x) || p(y))\right)$$

low entropy high entropy

low entropy points to $p(y|x)$ and high entropy points to $p(y)$

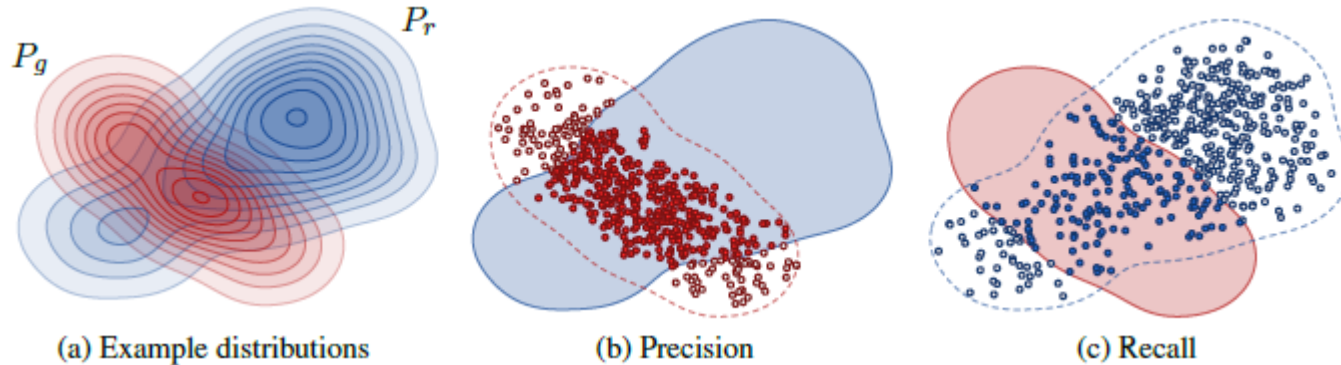
2. Fréchet Inception Distance (FID) [[Heusel-2017](#)]

- Two datasets – synthetic, real
- Each sample is “embedded” (features of the penultimate layer)
- Fit Gaussians $(\mu, \Sigma), (\mu', \Sigma')$

$$\text{FID} = \|\mu - \mu'\|_2^2 + \text{Tr}\left(C + C' - 2(CC')^{1/2}\right)$$

How to Measure Quality of Generative Models?

3. Precision-Recall for distributions [[Kynkäänniemi-2019](#)]



$$f(\phi, \Phi) = \begin{cases} 1, & \text{if } \|\phi - \phi'\|_2 \leq \|\phi' - \text{NN}_k(\phi', \Phi)\|_2 \text{ for at least one } \phi' \in \Phi \\ 0, & \text{otherwise,} \end{cases}$$

$$\text{precision}(\Phi_r, \Phi_g) = \frac{1}{|\Phi_g|} \sum_{\phi_g \in \Phi_g} f(\phi_g, \Phi_r)$$

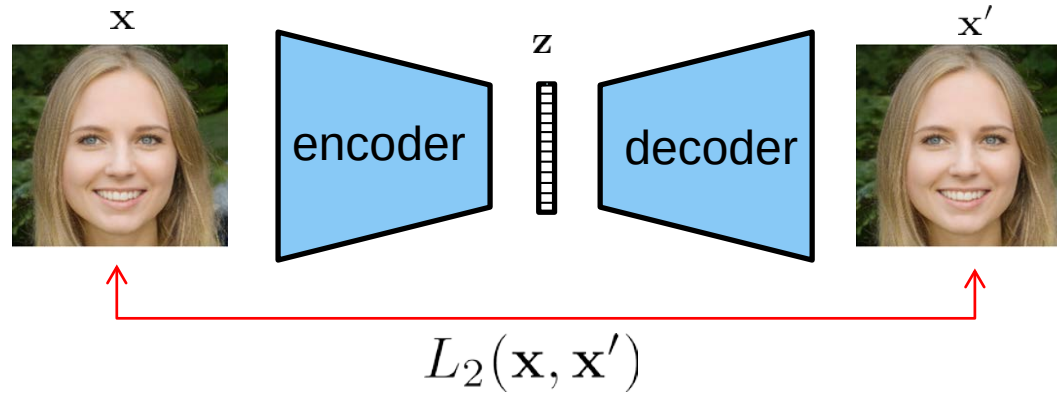
$$\text{recall}(\Phi_r, \Phi_g) = \frac{1}{|\Phi_r|} \sum_{\phi_r \in \Phi_r} f(\phi_r, \Phi_g)$$



Varioational AutoEncoders (VAEs)

Variational Autoencoders

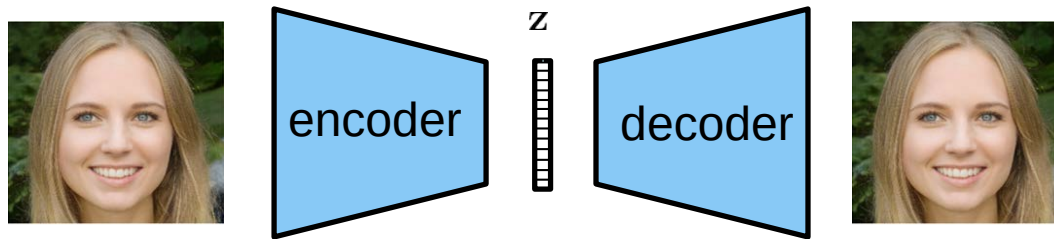
■ Autoencoder



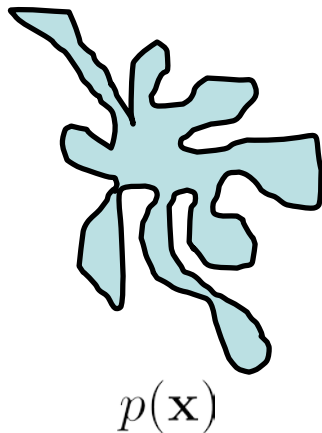
- Encoder-decoder architecture
- Bottleneck layer
- Data compression
- Several variants – masked autoencoder, denoising autoencoder
- Can we generate samples?

Variational Autoencoders

- Standard autoencoders have an unstructured latent space
 - Generating random latent and passing through decoder will not produce a meaningful sample
 - We need the latent vectors follow a “reasonable distribution”
 - e.g., Gaussian $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$

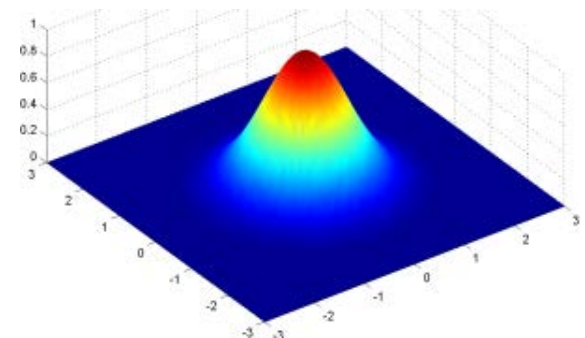


data distribution



$$p(\mathbf{z} | \mathbf{x})$$

latent distribution

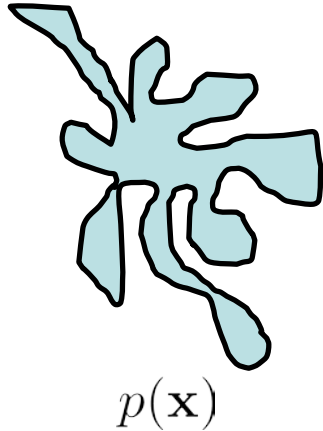


$$p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$$

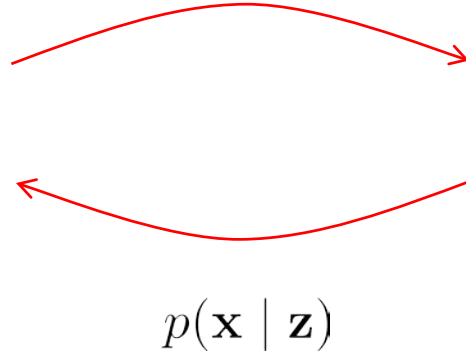
$$p(\mathbf{x} | \mathbf{z})$$

Variational Autoencoder

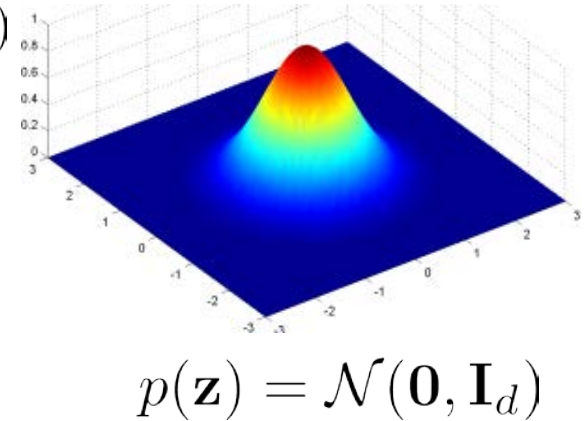
data distribution



$$p(\mathbf{z} | \mathbf{x}) \approx q(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mu, \sigma)$$



latent distribution



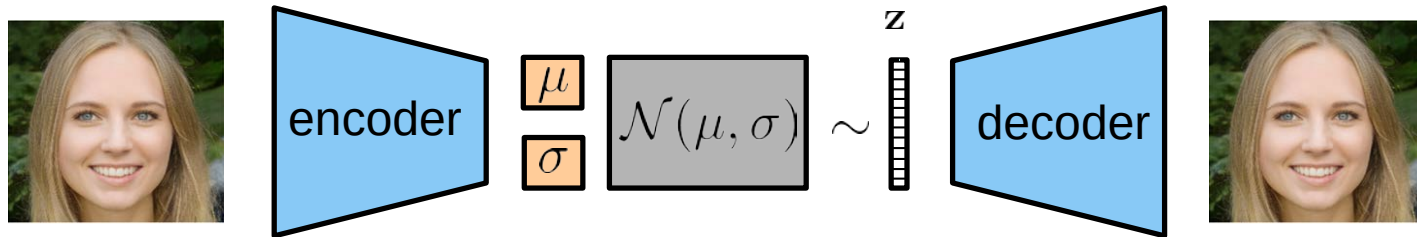
- Loss function: “Evidence Lower Bound” (lower bound on the log-likelihood)

$$\text{ELBO}(x) = \mathbf{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z))$$

- First term: reconstruction loss
- Second term: regularization loss

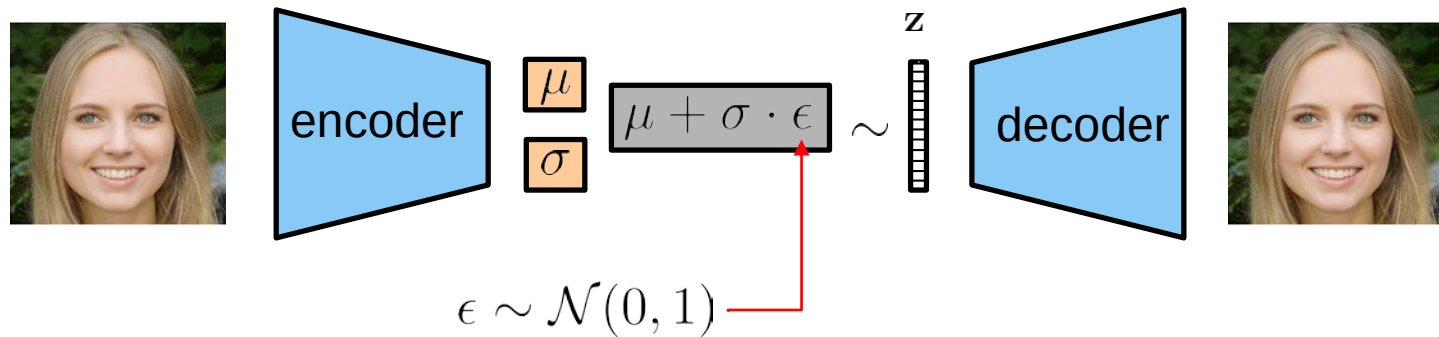


Variational Autoencoder



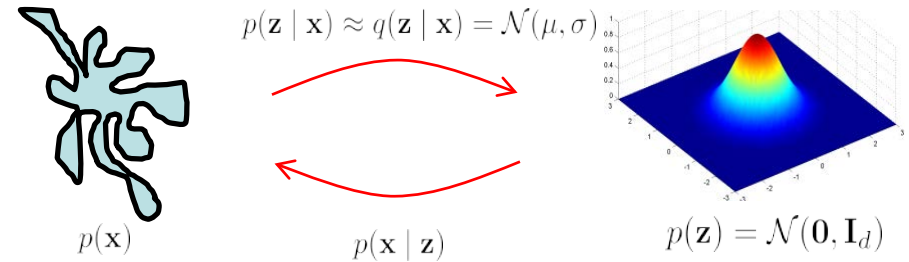
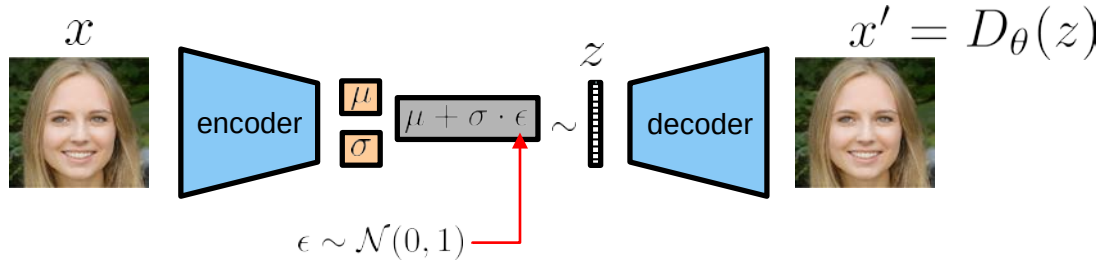
- A data point is mapped to a distribution
- How can we back-propagate through a stochastic node?

Variational Autoencoder



- A data point is mapped to a distribution
- How can we back-propagate through a stochastic node?
- We cannot => reparametrization trick
 - Encoder predicts (μ, σ) , ϵ is sampled
 - End-to-end differentiable

Variational Autoencoders



$$\text{ELBO}(x) = \underbrace{\mathbf{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)]}_{\text{First term}} - \underbrace{\text{KL}(q_{\phi}(z | x) \| p(z))}_{\text{Second term}}$$

- First term => L2 loss

$$p_{\theta}(x|z) = K e^{-\left(x - D_{\theta}(z)\right)^2}$$

$$\sum_i (x_i - x'_i)^2$$

- Second term (closed form)

$$\text{KL}(q \| p) = (\mathcal{N}(\mu, \sigma^2 \mathbf{I}) \| \mathcal{N}(\mathbf{0}, \mathbf{I}))$$

$$\sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1)$$

Variational Autoencoders – Summary



PROS:

- Stable training
 - Optimization is straightforward and converges easily
 - New data can be generated by sampling from Gaussian distribution
 - Fast sampling
- Easy inversion (projecting an image to a latent space)
 - Encoder is available
- Smooth, continuous latent space
 - Interpolations are meaningful, latent directions often carry semantics

CONS:

- Often “blurry” reconstruction (L2-loss)
- Not state-of-the art for high-quality samples (low resolution only)
- Sometimes, the Gaussian regularizer hurts the reconstruction
- The bottleneck dimensionality must be tuned



Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs)



"Generative Adversarial Networks is the **most interesting idea in the last ten years** in machine learning."

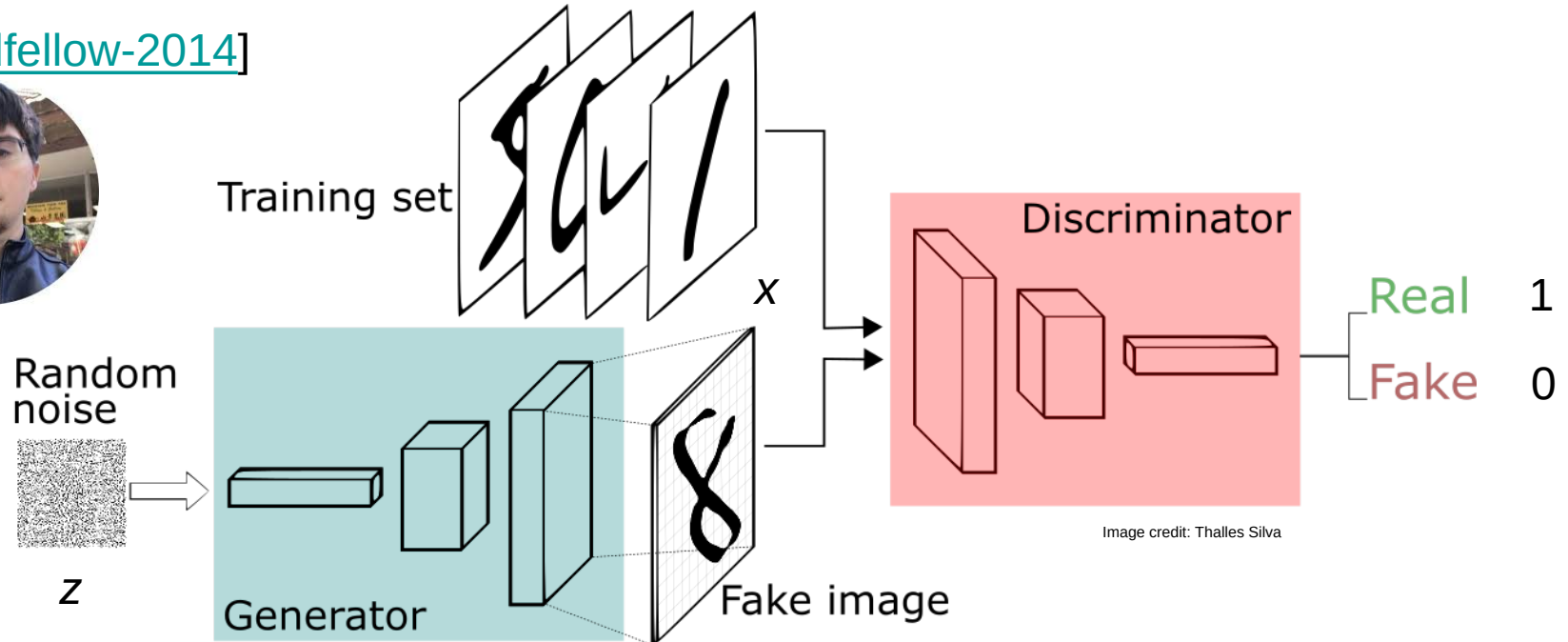
Yann LeCun, Director, Facebook AI





Generative Adversarial Networks (GANs)

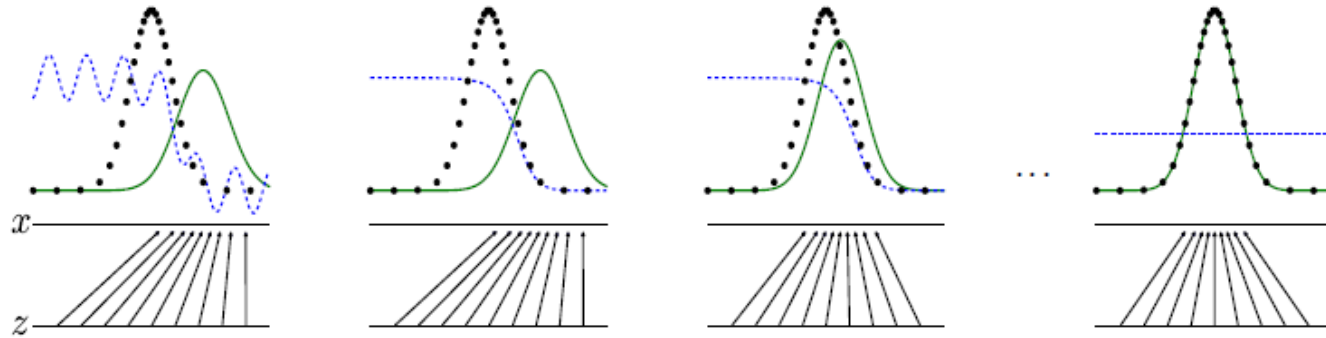
[Goodfellow-2014]



- Two networks: Generator $G: N(0,1)^k \rightarrow X$, Discriminator $D: X \rightarrow [0,1]$
- Min max game between G and D when training
 - The discriminator tries to distinguish generated and real samples
 - The generator tries to fool the discriminator

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Generative Adversarial Networks (GANs)



- Seems to capture the image manifold
 - Smooth transitions when interpolating in the latent space

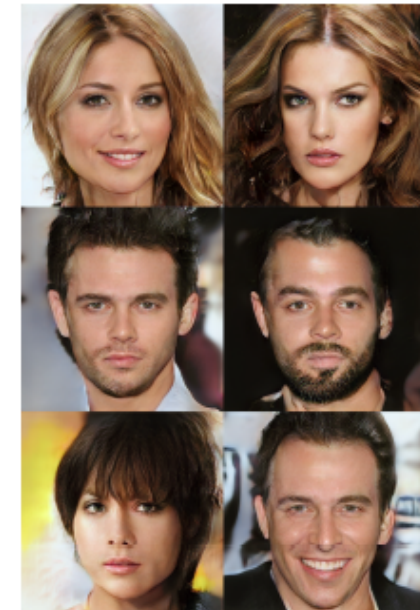
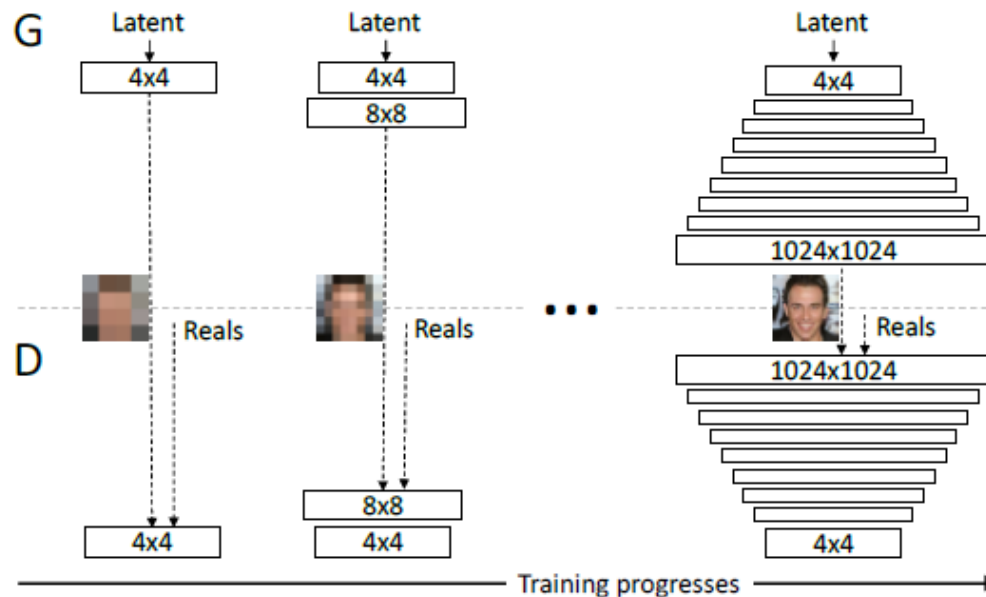
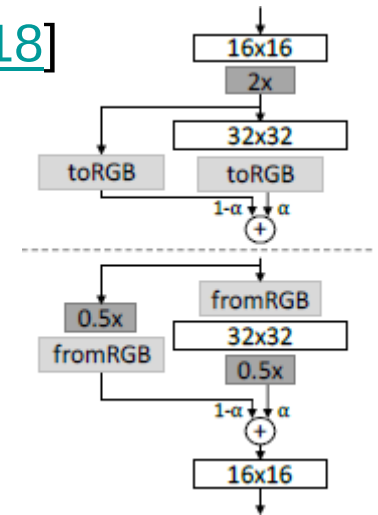


- However:
 - The training is fragile (alternating optimization), mode collapse
 - Did not work well for high-resolution (until ~2018)



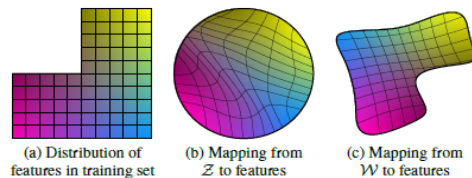
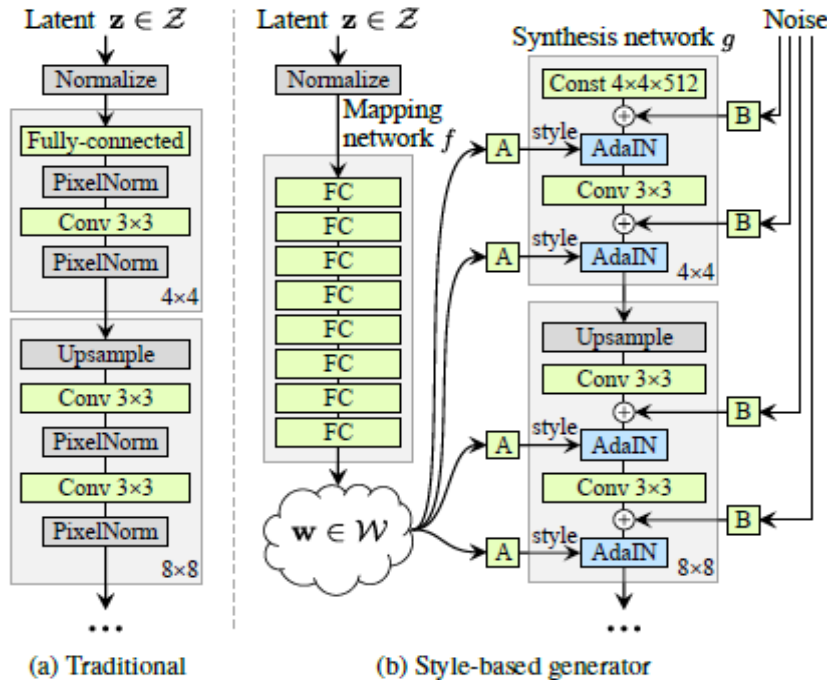
High resolution GANs

- Synthesis of 1024x1024 face images [[Nvidia-ProGAN-2018](#)]
- Trained from CelebA-HQ dataset 30k images
- Progressive training
 - Complete GAN for low-resolution (4x4)
 - Upsample, concatenate with res-net connections
 - Train everything end-to-end



StyleGAN

- StyleGAN [[Karras-2019](#)] (Nvidia)



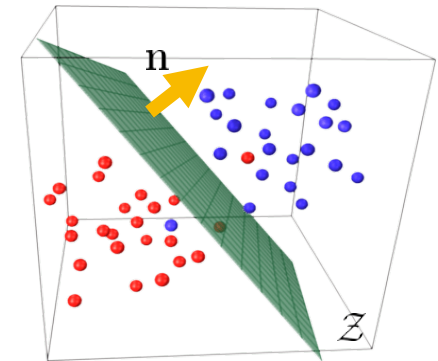
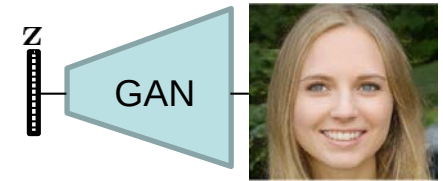
- Multi-layer style transfer, training from 70k Flickr dataset, “[hyper-realistic](#)”

- Follow-up paper [[Nvidia-2020](#), [Nvidia-2021](#), [Nvidia-2022](#)]

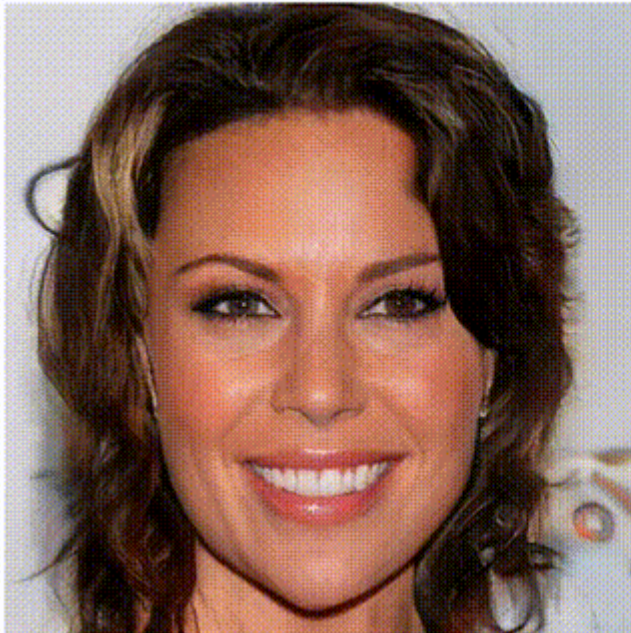


GAN – latent space manipulation

- Every z from input distribution gives a realistic image
- Finding semantic direction in the latent vector space
 - Train a linear binary classifier on labeled set (z_i, y_i)
 - Normal of the discriminative hyperplane is the semantic direction
- Semantic Editing / “Manipulation” $z = z_0 + \alpha n$



INSTRUCTION: press +/- to adjust feature, toggle feature name to lock the feature



random face		
Male	Age	Skin_Tone
- +	- +	- +
Bangs	Hairline	Bald
- +	- +	- +
Big_Nose	Pointy_Nose	Makeup
- +	- +	- +
Smiling	Mouth_Open	Wavy_Hair
- +	- +	- +
Beard	Goatee	Sideburns
- +	- +	- +
Blond_Hair	Black_Hair	Gray_Hair
- +	- +	- +
Eyeglasses	Earrings	Necktie
- +	- +	- +



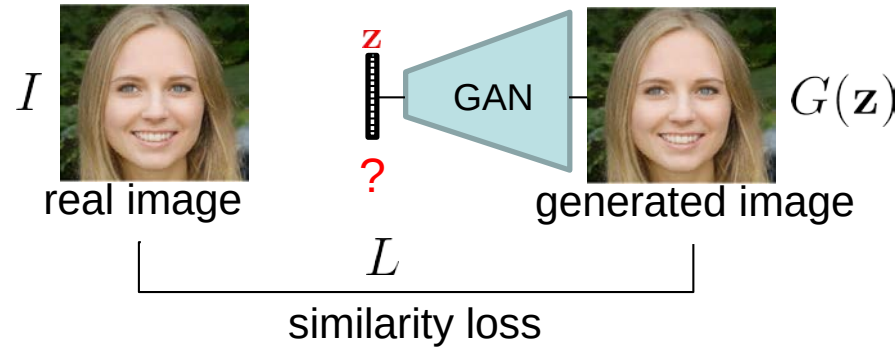
[demo]

[Abdal-SIGGRAPH-2021]



GAN Inversion (projection)

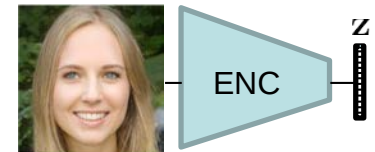
- Given an input real image I , find the latent code that generate the image



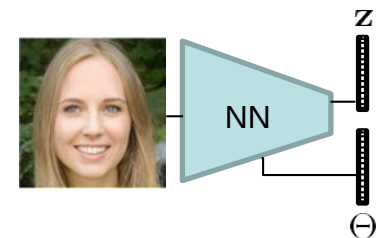
- Many approaches:

- Direct optimization
 - Works, but slow (~10s)
- Encoders [[Alaluf-2022](#), [Tov-2021](#)]
 - Very fast, but less accurate
- Pivotal tuning [[Roich-2021](#)]
 - Optimizes the GAN model parameters
- Hyper-networks [[Alaluf-2022](#)]
 - NN which adjust GAN model parameters
- Piece-wise inversion [[Šubrtová-2022](#)]
 - More degrees of freedom, very accurate, but slow

$$\min_{\mathbf{z}} L(I, G(\mathbf{z}))$$

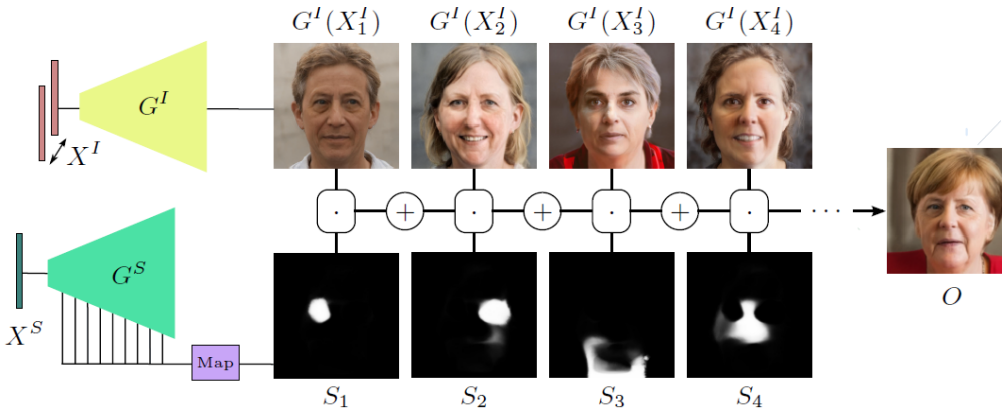


$$\min_{\mathbf{z}, \Theta} L(I, G(\mathbf{z}, \Theta))$$



GAN Inversion (projection)

- Chunky GAN – piecewise image inversion [[Šubrtová-2022](#)]



$$O(X^I, S) = \sum_{i=1}^n G^I(X_i^I) \cdot S_i.$$

$$\min_{X^S, X^I} \mathcal{L}_{\text{LPIPS}}(I, \sum_{i=1}^n G^I(X_i^I) \cdot G^S(X^S)_i) + \lambda_{\text{reg}} \sum_{i=1}^n \|X_i^I - X_\mu^I\|_2^2.$$

- Partial inversion, Interactive editing

ChunkyGAN: Real Image Inversion via Segments



Real Photo Editing

- Image $\rightarrow$ crop face $\rightarrow$ project to SG latent space $\rightarrow$ manipulate $\rightarrow$ blend back to the original image

[[Petrzelkova-2021](#)]



(a) Original photo



(b) Gender +2.7



(a) Original photo



Age +2.3, eyeglasses -1.2, gender +2.1



(c) Age -0.6, gender -0.7, smile



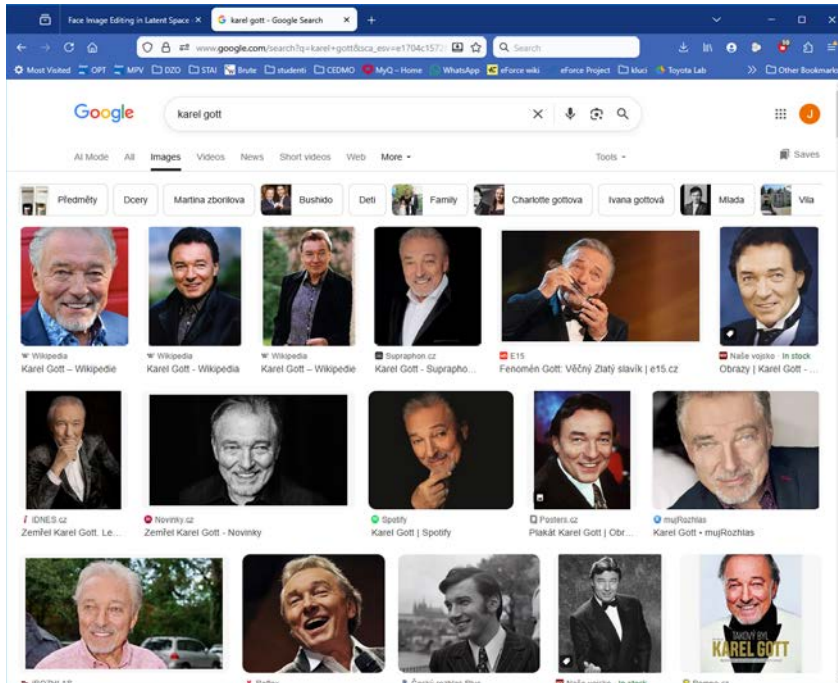
(c) Eyeglasses +2.5, gender -0.7



(d) Gender +1.7

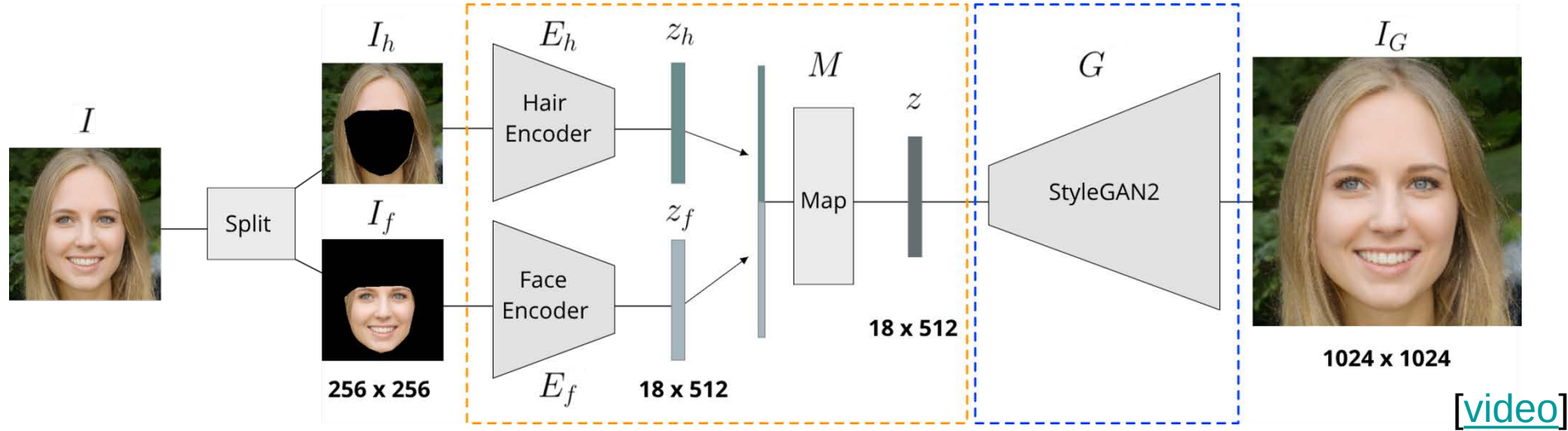
Facial Time Lapse Video

- Fully automating aging time laps video [[Vereš-2022](#)]
- Celebrity name → search engine → images → verify identity, sort by age → project to SG → interpolate smoothly in the latent space

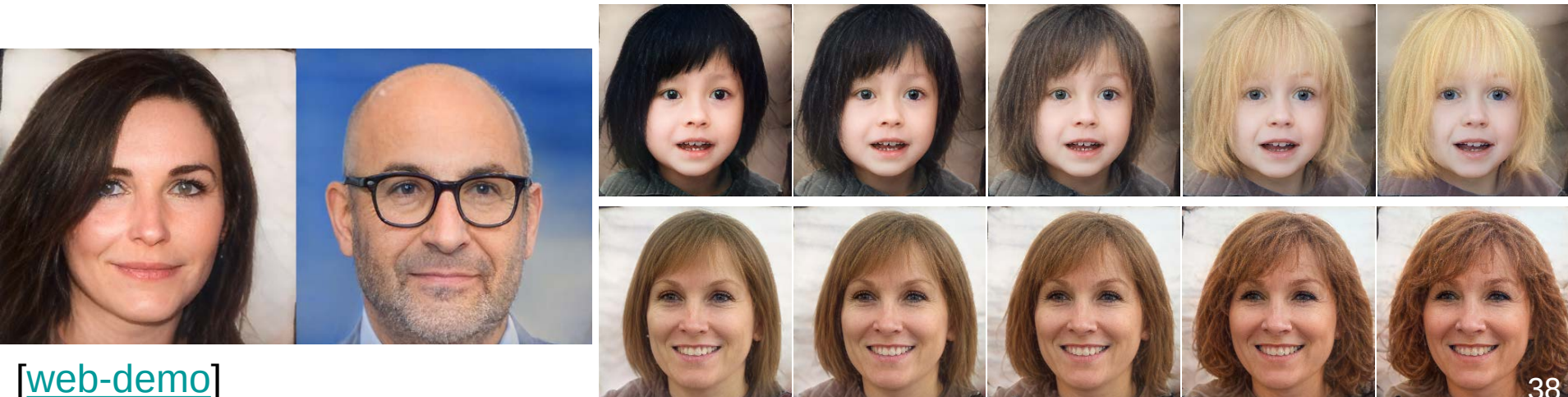


Hairstyle Transfer using StyleGAN

- Fully automatic hairstyle transfer, unaligned portraits [[Šubrtová-FG-2021](#)]



- Basic idea: Train two encoders (Hair, face) + fixed StyleGAN decoder
- Hairstyle interpolation, Editing in hairstyle latent space



Text-based Image Manipulation

- StyleCLIP [[Patashnik-2021](#)]
 - Text-Driven Manipulation of StyleGAN Imagery
 - Latent code manipulation driven by [CLIP](#) text-image similarity



Input

“Beyonce”

“A woman
without makeup”

“Elsa from
Frozen”



Input

“A man with a
beard”

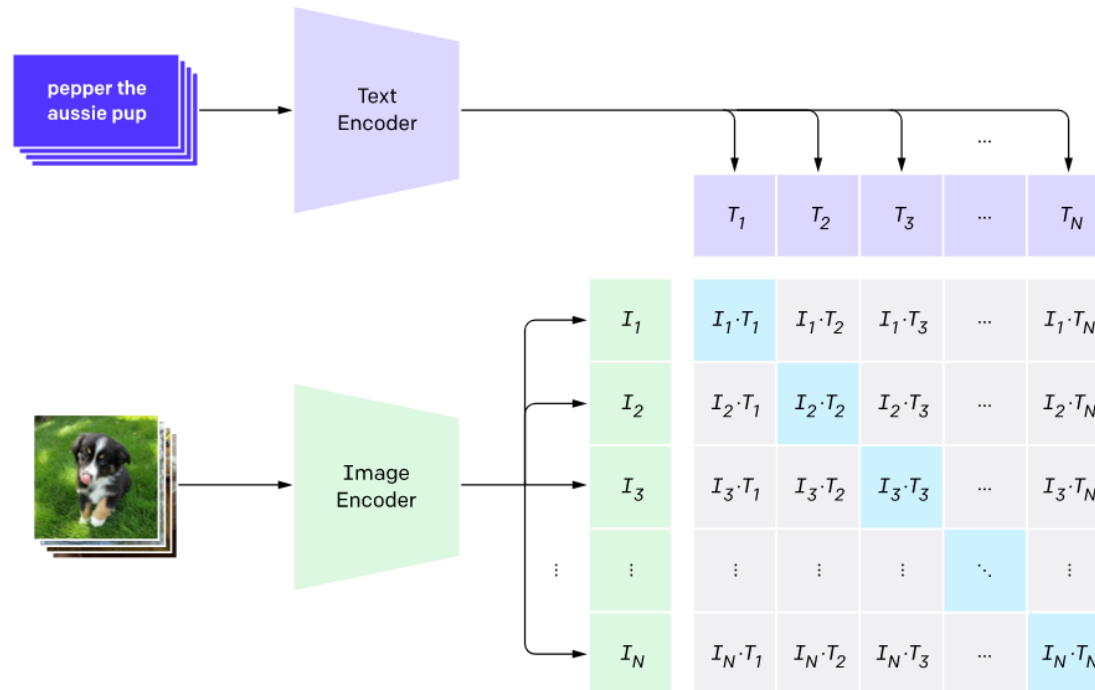
“A blonde man”

“Donald Trump”

$$\arg \min_{w \in \mathcal{W}+} D_{\text{CLIP}}(G(w), t) + \lambda_{\text{L2}} \|w - w_s\|_2 + \lambda_{\text{ID}} \mathcal{L}_{\text{ID}}(w)$$

CLIP – Connecting Text and Images (recap)

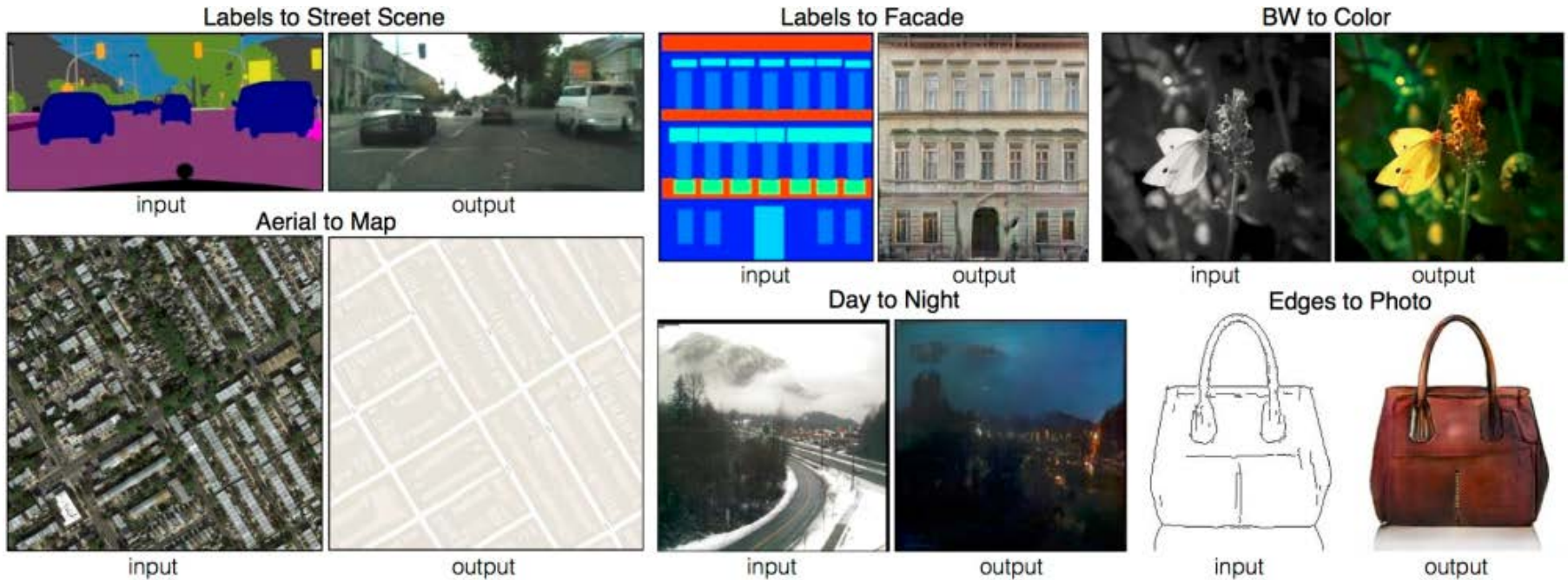
- CLIP [[Radford-2021](#)] by OpenAI
 - “*Contrastive Language–Image Pre-training*”
 - Learn joint text-image embedding => Text-image (cosine) similarity
 - Learned from 400M WebImageText (WIT) dataset



- Zero-shot prediction (on par with Resnet on ImageNET benchmark)
 - Loop over ImageNET-classes: $\max \text{CLIP}(E_T(\text{"A photo of a <class>"}), E_I(I))$
- Trained [model](#) publicly available

Image to Image Translation

- Transfer image between domains [[Isola-2017](#)]



- Many applications [[pix2pix](#)], Super-resolution [[Šubrtová-2018](#)]



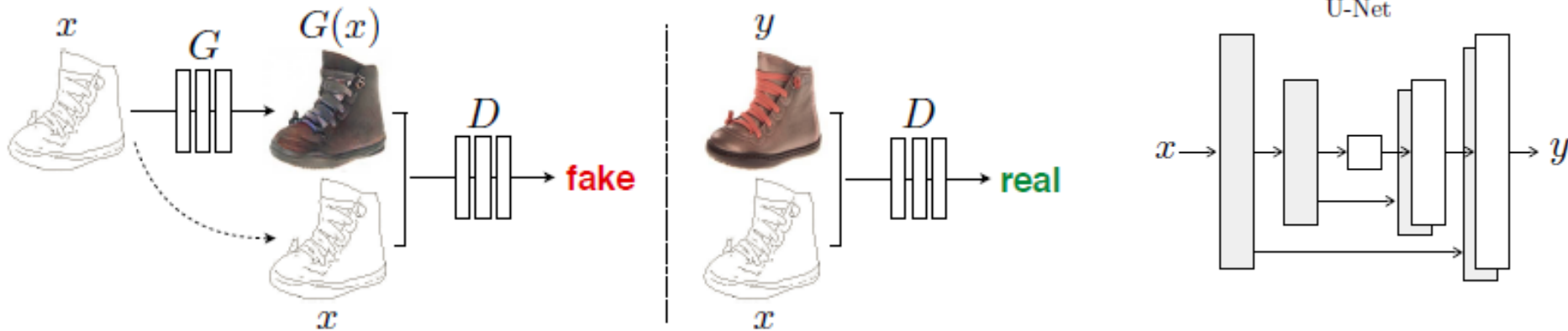
16x16

256x256 (predicted)

256x256 (ground-truth)

Image to Image Translation

- Combines fully convolutional net training with (conditional) GAN



$$G^* = \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G)$$

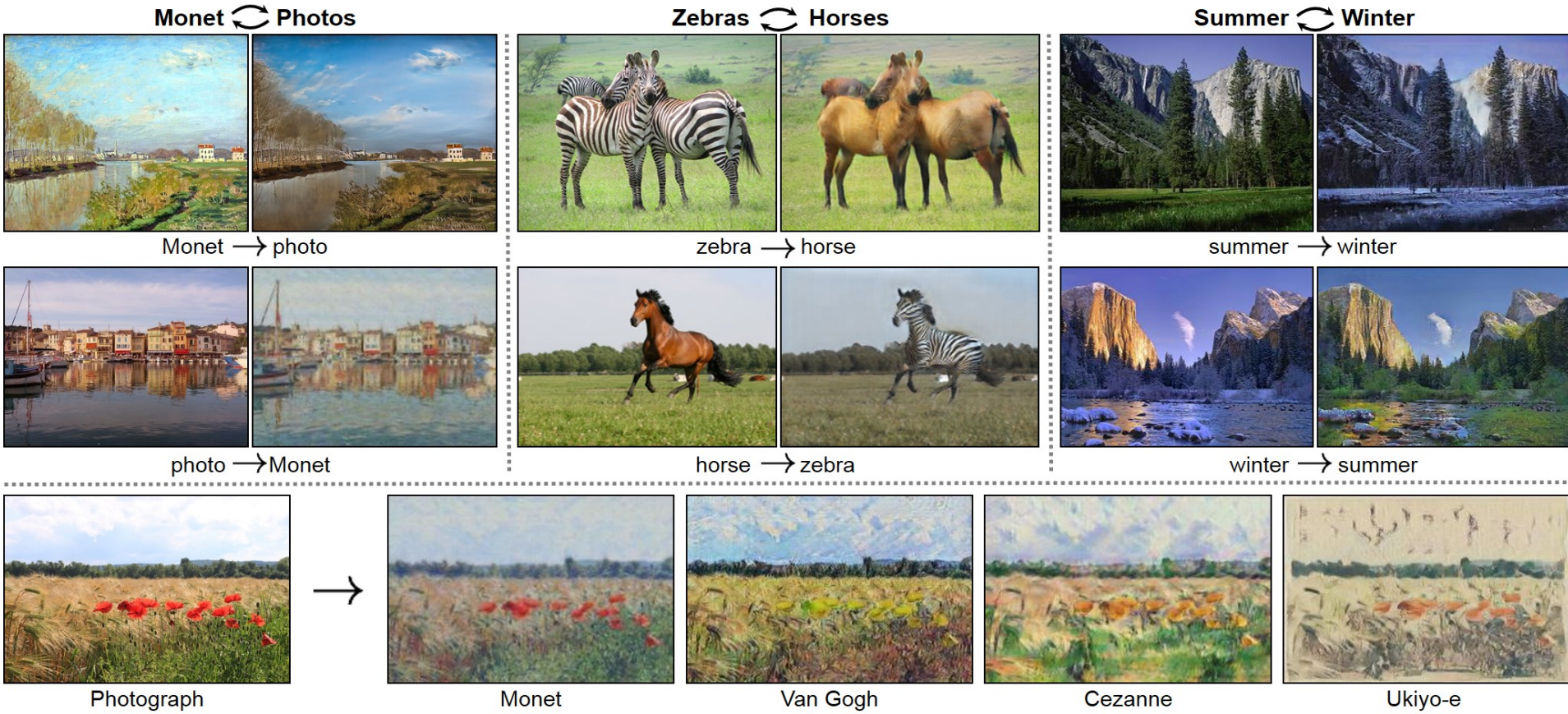
$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z} [\|y - G(x, z)\|_1]$$

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y} [\log D(x, y)] + \mathbb{E}_{x,z} [\log(1 - D(x, G(x, z)))]$$

- Difficulties with imposing variability (only via dropout when testing)
- Training needs pixel-to-pixel source and target image correspondences

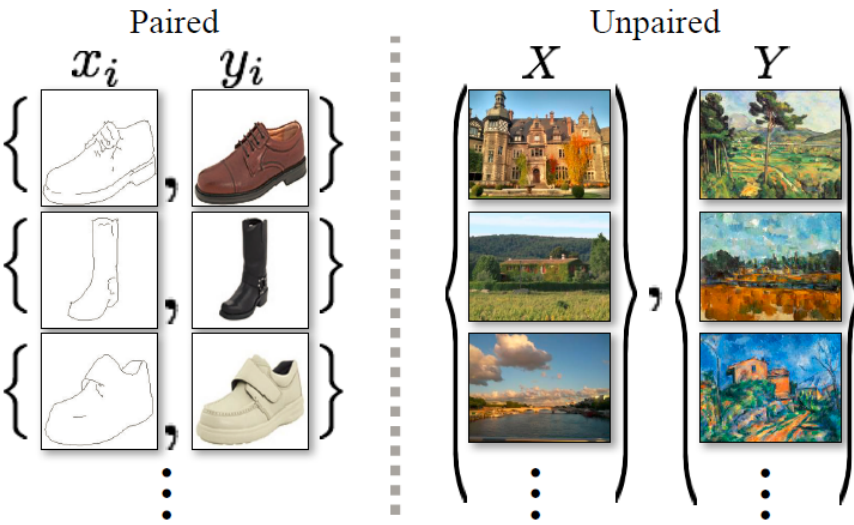
Cycle GAN

- Translating without pix-to-pix correspondences [[Zhu-2017](#)]



Cycle GAN

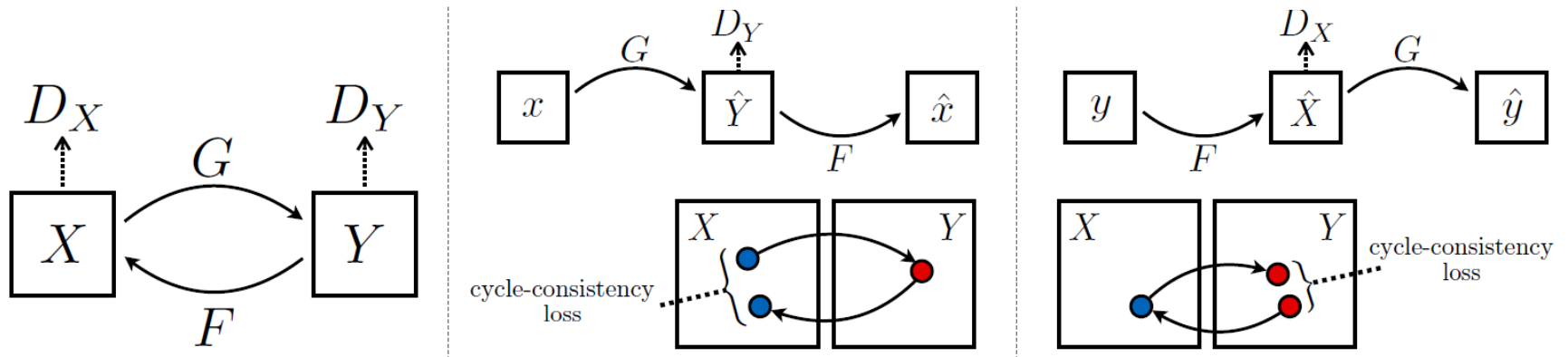
- Unpaired set of images to train the translation



$$\begin{aligned}\mathcal{L}(G, F, D_X, D_Y) = & \mathcal{L}_{\text{GAN}}(G, D_Y, X, Y) \\ & + \mathcal{L}_{\text{GAN}}(F, D_X, Y, X) \\ & + \lambda \mathcal{L}_{\text{cyc}}(G, F),\end{aligned}$$

$$\begin{aligned}\mathcal{L}_{\text{cyc}}(G, F) = & \mathbb{E}_{x \sim p_{\text{data}}(x)} [\|F(G(x)) - x\|_1] \\ & + \mathbb{E}_{y \sim p_{\text{data}}(y)} [\|G(F(y)) - y\|_1]\end{aligned}$$

- Cycle consistency



GANs - summary



PROS:

- High-quality samples
 - High-resolution, sharp, realistic images (far better than VAEs)
- Latent space often semantically meaningful
 - Interpolation works, semantic direction, linear manipulation
- Fast sampling
 - Single forward pass through the generator
- Trade-off between quality and diversity of data can be controlled

CONS:

- Training instability
 - Hard to train, minimax game instable, mode collapse (balancing generator/discriminator)
 - Sensitive to hyperparameters (and architecture changes)
- Hard to invert (project images to latent space)